

Week-1 Module

Part 1 :- Star Patterns

① Square

```

* * * * *
* * * * *
* * * * *
* * * * *
* * * * *

```

- Logic :-
- 1) Use two nested loops : an outer loop for rows, an inner loop for columns
 - 2) Outer loop runs from 1 to n
 - 3) Inner loop runs col from 1 to n, printing * each time
 - 4) After inner loop finishes, print a newline to move to the next row.

```

Code :- public class Square {
        public static void main (String [] args) {
            int n = 5;
            for (int row = 1; row ≤ n; row++) {
                for (int col = 1; col ≤ n; col++) {
                    System.out.println(" * ");
                }
                System.out.println();
            }
        }
    }

```

② Hollow Square

```

* * * * *
*           *
*           *
*           *
* * * * *

```

Logic :-

- 1) Same nested loop, an outer loop for rows and inner loop for columns.
- 2) for each cell (row, col) print * only if it's on the border - i.e., row = 1, row = n, col = 1 or col = n.
- 3) Otherwise, print a space, so inside stays hollow.

Spiral

```

Code :- public class HollowSquare {
    public static void main (String [] args) {
        int n = 5;
        for (int row = 1; row <= n; row++) {
            for (int col = 1; col <= n; col++) {
                if (row == 1 || row == n || col == 1 || col == n) {
                    System.out.print ("*");
                }
                else {
                    System.out.print (" ");
                }
            }
            System.out.println();
        }
    }
}

```

3) Right Triangle

```

*
* *
* * *
* * * *
* * * * *

```

logic :-

- 1) Outer loop runs from 1 to n.
- 2) Inner loop runs col from 1 to row (not to n!) - so row 1 prints 1 star, row 2 prints 2 stars and so on.
- 3) Print a newline after each row.

```

Code :- public class Triangle {
    public static void main (String [] args) {
        int n = 5;
        for (int row = 1; row <= n; row++) {
            for (int col = 1; col <= row; col++) {
                System.out.print ("*");
            }
            System.out.println();
        }
    }
}

```


④ Inverted Triangle

**

*

Logic:-

- 1) Outer loop runs row from 1 to n.
- 2) Inner loop runs col from 1 to $(n - \text{row} + 1)$
- this starts at n stars and shrinks by 1 each row.
- 3) Print a newline after each row

```
Code:- public class InvertedTriangle {
    public static void main (String [] args) {
        int n=5;
        for (int row=1; row ≤ n; row++) {
            for (int col=1; col ≤ n-row+1; col++) {
                System.out.print ("*");
            }
            System.out.println(" ");
        }
    }
}
```

⑤ Pyramid

*

Logic:-

- 1) Outer loop runs row from 1 to n.
- 2) First, print $(n - \text{row})$ leading spaces to push stars towards center.
- 3) then print $(2 * \text{row} - 1)$ stars - this gives odd sequence 1, 3, 5, 7, 9 and print newline.

```
Code:- public class Pyramid {
    public static void main (String [] args) {
        int n=5;
        for (int row=1; row ≤ n; row++) {
            for (int sp=1; sp ≤ n-row; sp++) {
                System.out.print (" ");
            }
            System.out.print ("*");
        }
    }
}
```

Spiral

```

for (int star=1; star ≤ 2*row-1; star++) {
    System.out.print(" ");
}
System.out.println();
}
}
}

```

⑥ Diamond

```

  *
 * * *
* * * * *
* * * * * * *
 * * * * *
  * * *
   *

```

Logic :-

- 1) A diamond is a pyramid (growing half) immediately followed by upside down pyramid. 9 rows total for $n=5$.
- 2) Use one loop variable i from 1 to $(2n-1)$
- 3) Compute a level k if $i \leq n$, then $k=i$ otherwise $k=2n-i$ (shrinking half)
- 4) Print $(n-k)$ spaces, then $(2k-1)$ stars - exactly the same formula as pyramid, driven by k .

Code :-

```

public class Diamond {
    public static void main (String [] args) {
        int n=5;
        for (int i=1; i ≤ n*2-1; i++) {
            int k = (i ≤ n) ? i : 2*n-i;
            for (int sp=1; sp ≤ n-k; sp++) {
                System.out.print(" ");
            }
            for (int star=1; star ≤ 2*k-1; star++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}

```


⑦ Half-diamond

```

*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*

```

logic :-

- 1) Same "grow then shrink" idea as diamond but left-aligned and increasing by 1 star per row instead of 2.
- 2) Use i from 1 to $(2n-1)$, with the same level formula: $K = (i \leq n) ? i : 2n - i$
- 3) Print K stars (no leading spaces needed since its flush out).

```

Code:- public class HalfDiamond {
    public static void main(String[] args) {
        int n = 5;
        for (int i = 1; i <= 2 * n - 1; i++) {
            int k = (i <= n) ? i : 2 * n - i;
            for (int star = 1; star <= k; star++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}

```

⑧ Half Diamond Inverted

```

* * * * *
* * * *
* * *
* *
*

```

logic :-

- 1) Same logic as Inverted Triangle: outer loop row from 1 to n
- 2) Inner loop prints $(n - \text{row} + 1)$ stars, decreasing each row.

```

Code :- public class HalfDiamondInverted {
        public static void main (String [] args) {
            int n=5;
            for (int row=1; row ≤ n; row++) {
                for (int star=1; star ≤ n-row+1; star++) {
                    System.out.print ("*");
                }
                System.out.println();
            }
        }
    }

```

Ex 2:- Number Patterns

① Square

```

1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1

```

Logic :-

- 1) Same nested-loop as the star square
- 2) Outer loop row from 1 to n, inner loop col from 1 to n.
- 3) Instead of printing *, print digit "1" every time.

```

Code :- public class NumberSquare {
        public static void main (String [] args) {
            int n=5;
            for (int row=1; row ≤ n; row++) {
                for (int col=1; col ≤ n; col++) {
                    System.out.print ("1");
                }
                System.out.println();
            }
        }
    }

```


② Incrementing

```

1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5

```

Logic:-

- 1) Outer loop row from 1 to n - every row looks identical.
- 2) Inner loop col from 1 to n, printing the value of col each time.

```

Code:- public class Incrementing {
    public static void main (String [] args) {
        int n = 5;
        for (int row = 1; row <= n; row++) {
            for (int col = 1; col <= n; col++) {
                System.out.print (col + " ");
            }
            System.out.println ();
        }
    }
}

```

③ Right Triangle

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

```

Logic:-

- 1) Outer loop row from 1 to n
- 2) Inner loop col from 1 to row, printing the value of col each time - so each row counts up to its row number.

```

Code:- public class RightTriangle {
    public static void main (String [] args) {
        int n = 5;
        for (int row = 1; row <= n; row++) {
            for (int col = 1; col <= row; col++) {
                System.out.print (col + " ");
            }
            System.out.println ();
        }
    }
}

```

Spiral

(4) Inverted Diamond

```

1 1 1 1 1
2 2 2 2
3 3 3
4 4
5

```

Logic :-

- 1) Outer loop row from 1 to n.
- 2) The digit printed on row is (row+2)
- 3) The no. of times it printed is (n - row + 1)
- 4) Inner loop runs col from 1 to count, printing value each time.

Code :-

```

public class InvertedDiamond {
    public static void main (String [] args) {
        int n = 5;
        for (int row = 1; row ≤ n; row++) {
            int value = row + 2;
            int count = n - row + 1;
            for (int col = 1; col ≤ count; col++) {
                System.out.print (value + " ");
            }
            System.out.println();
        }
    }
}

```

(5) Sandwich pattern

```

3 3 3 3 3
4 4 4
5
4 4 4
3 3 3 3 3

```

Logic :-

- 1) Outer loop row from 1 to n.
- 2) Find how far the current row is from the nearer edge : distance = min (row - 1, n - row)
- 3) count = n - 2 * distance (shrinks towards middle)
- 4) value = (n + 1) / 2 + distance - the digit to print.
- 5) Print distance * 2 leading spaces, then print value repeated count times.

Spiral

Code:-

```

public class sandwichPattern {
    public static void main (String [] args) {
        int n = 5;
        for (int row = 1; row ≤ n; row++) {
            int distance = Math.min(row - 1, n - row);
            int count = n - 2 * distance;
            int value = (n + 1) / 2 + distance;
            for (int sp = 1; sp ≤ distance * 2; sp++) {
                System.out.print(" ");
            }
            for (int col = 1; col ≤ count; col++) {
                System.out.print(value + " ");
            }
            System.out.println();
        }
    }
}

```

⑥ Incrementing Diamond

```

      3
    4 4
  5 5 5
6 6 6 6
7 7 7 7 7
  6 6 6 6
    5 5 5
      4 4
        3

```

Logic:-

- 1) Same "grow then shrink" shape as the star Diamond - 9 rows for $n=5$
- 2) Use i from 1 to $(2n-1)$, with level $K = (i \leq n)$
 $? i: 2n - i$ (gives 1, 2, 3, 4, 5, 4, 3, 2, 1)
- 3) The digit printed is value = $K + 2$, where $K = n$, print largest digit.
- 4) Print $(n - K) * 2$ leading spaces to center the row, print value repeated K times.

Code:-

```

public class IncrementingDiamond {
    public static void main (String [] args) {
        int n=5;
        for (int i=1; i ≤ 2*n-1; i++) {
            int k = (i ≤ n) ? i : 2*n-i;
            int value = k+2;
            for (int sp=1; sp ≤ (n-k)*2; sp++) {
                System.out.print(" ");
            }
            for (int col=1; col ≤ k; col++) {
                System.out.print (value + " ");
            }
            System.out.println();
        }
    }
}

```


Square.java > Language Support for Java(TM) by Red Hat > Square > main(String[])

```

1  public class Square {
    Run | Debug | Run main | Debug main
2      public static void main(String[] args) {
3          int n = 5;
4          for (int row = 1; row <= n; row++) {
5              for (int col = 1; col <= n; col++) {
6                  System.out.print(s: "+");
7              }
8              System.out.println();
9          }
10     }
11 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

+ ▾ ... | [] X

● PS D:\oops with java> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Bhavya aggarwal\AppData\Roaming\Code\User\workspaceStorage\cac716e0002c014cf17c54c283732460\redhat.java\jdt_ws\oops with java_5dfb3bf8\bin' 'Square'

○ PS D:\oops with java>

powersh... ⚠

Run: Square

HollowSquare.java > Java > HollowSquare

```
1 public class HollowSquare {  
    Run | Debug | Run main | Debug main  
2     public static void main(String[] args) {  
3         int n = 5;  
4         for (int row = 1; row <= n; row++) {  
5             for (int col = 1; col <= n; col++) {  
6                 if (row == 1 || row == n || col == 1 || col == n) {  
7                     System.out.print(s: "");  
8                 } else {  
9                     System.out.print(s: " ");  
10                }  
11            }  
12            System.out.println();  
13        }  
14    }  
15 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Run: HollowSquare + ▢ ▢ ... | [] X

```
PS D:\oops with java> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-  
cp' 'C:\Users\Bhavya aggarwal\AppData\Roaming\Code\User\workspaceStorage\cac716e0002c014cf17c54c283732460\redhat.java\jdt_ws\oops with j  
● ava_5dfb3bf8\bin' 'HollowSquare'
```

```
*****  
*   *  
*   *  
*   *  
*****
```

○ PS D:\oops with java>

Triangle.java > ...

```
1 public class Triangle {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
2         int n = 5;
3         for (int row = 1; row <= n; row++) {
4             for (int col = 1; col <= row; col++) {
5                 System.out.print(s: "");
6             }
7             System.out.println();
8         }
9     }
10 }
11
12
13
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Run: Triangle + ▼ □ ✕ ... | {} X

● PS D:\oops with java> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Bhavya aggarwal\AppData\Roaming\Code\User\workspaceStorage\cac716e0002c014cf17c54c283732460\redhat.java\jdt_ws\oops with java_5dfb3bf8\bin' 'Triangle'

*

**

○ PS D:\oops with java>

InvertedTriangle.java > Java > InvertedTriangle

```
1 public class InvertedTriangle {  
    Run | Debug | Run main | Debug main  
2     public static void main(String[] args) {  
3         int n = 5;  
4         for (int row = 1; row <= n; row++) {  
5             for (int col = 1; col <= n - row + 1; col++) {  
6                 System.out.print(s: "**");  
7             }  
8             System.out.println();  
9         }  
10    }  
11  
12
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Run: InvertedTriangle + ▾ □ ✕ | □ ✕

```
● PS D:\oops with java> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-  
cp' 'C:\Users\Bhavya aggarwal\AppData\Roaming\Code\User\workspaceStorage\cac716e0002c014cf17c54c283732460\redhat.java\jdt_ws\oops with j  
ava_5dfb3bf8\bin' 'InvertedTriangle'
```

```
*****  
****  
***  
**  
*
```

```
○ PS D:\oops with java>
```

Pyramid.java > Java > Pyramid

```
1 public class Pyramid {  
    Run | Debug | Run main | Debug main  
2     public static void main(String[] args) {  
3         int n = 5;  
4         for (int row = 1; row <= n; row++) {  
5             for (int sp = 1; sp <= n - row; sp++) {  
6                 System.out.print(s: " ");  
7             }  
8             for (int star = 1; star <= 2 * row - 1; star++) {  
9                 System.out.print(s: "*");  
10            }  
11            System.out.println();  
12        }  
13    }  
14 }  
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Run: Pyramid +   ... |  X

```
PS D:\oops with java> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-  
cp' 'C:\Users\Bhavya aggarwal\AppData\Roaming\Code\User\workspaceStorage\cac716e0002c014cf17c54c283732460\redhat.java\jdt_ws\oops with j  
● ava_5dfb3bf8\bin' 'Pyramid'
```

```
*  
***  
*****  
*****  
*****
```

○ PS D:\oops with java>

Diamond.java > Java > Diamond > main(String[] args)

```
1 public class Diamond {
2     public static void main(String[] args) {
3         int n = 5;
4         for (int i = 1; i <= 2 * n - 1; i++) {
5             int k = (i <= n) ? i : 2 * n - i;
6             for (int sp = 1; sp <= n - k; sp++) {
7                 System.out.print(" ");
8             }
9             for (int star = 1; star <= 2 * k - 1; star++) {
10                System.out.print("*");
11            }
12            System.out.println();
13        }
14    }
15 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Run: Diamond + ▾ □ ✕ ... | {} X

```
PS D:\oops with java> d:; cd 'd:\oops with java'; & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDe
tailsInExceptionMessages' '-cp' 'C:\Users\Bhavya aggarwal\AppData\Roaming\Code\User\workspaceStorage\cac716e0002c014cf17c54c283732460\re
dhat.java\jdt_ws\oops with java_5dfb3bf8\bin' 'Diamond'
```

*

PS D:\oops with java>

HalfDiamond.java > Java > HalfDiamond

```
1 public class HalfDiamond {  
    Run | Debug | Run main | Debug main  
2     public static void main(String[] args) {  
3         int n = 5;  
4         for (int i = 1; i <= 2 * n - 1; i++) {  
5             int k = (i <= n) ? i : 2 * n - i;  
6             for (int star = 1; star <= k; star++) {  
7                 System.out.print(s: " ");  
8             }  
9             System.out.println();  
10        }  
11    }  
12 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Run: HalfDiamond + ▾ □ ✕ ... | □ ✕

```
PS D:\oops with java> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-  
cp' 'C:\Users\Bhavya aggarwal\AppData\Roaming\Code\User\workspaceStorage\cac716e0002c014cf17c54c283732460\redhat.java\jdt_ws\oops with j  
ava_5dfb3bf8\bin' 'HalfDiamond'
```

```
*****  
*****  
*****  
*****  
*****  
*****  
*****
```

○ PS D:\oops with java>

Incrementing.java > Language Support for Java(TM) by Red Hat > Incrementing

```
1 public class Incrementing {  
    Run | Debug | Run main | Debug main  
2     public static void main(String[] args) {  
3         int n = 5;  
4         for (int row = 1; row <= n; row++) {  
5             for (int col = 1; col <= n; col++) {  
6                 System.out.print(col + " ");  
7             }  
8             System.out.println();  
9         }  
10    }  
11  
12
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Run: Incrementing + ▼ □ ✕ | □ ✕

```
PS D:\oops with java> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-  
cp' 'C:\Users\Bhavya aggarwal\AppData\Roaming\Code\User\workspaceStorage\cac716e0002c014cf17c54c283732460\redhat.java\jdt_ws\oops with j  
ava_5dfb3bf8\bin' 'Incrementing'  
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5  
PS D:\oops with java>
```


RightTriangle.java > ...

```
1 public class RightTriangle {  
2     Run | Debug | Run main | Debug main  
3     public static void main(String[] args) {  
4         int n = 5;  
5         for (int row = 1; row <= n; row++) {  
6             for (int col = 1; col <= row; col++) {  
7                 System.out.print(col + " ");  
8             }  
9             System.out.println();  
10        }  
11    }  
12  
13
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Run: RightTriangle + ▾ □ 🗑 ... | [] X

● PS D:\oops with java> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Bhavya aggarwal\AppData\Roaming\Code\User\workspaceStorage\cac716e0002c014cf17c54c283732460\redhat.java\jdt_ws\oops with java_5dfb3bf8\bin' 'RightTriangle'

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```

○ PS D:\oops with java>

SandwichPattern.java > ...

```

1  public class SandwichPattern {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
2      int n = 5;
3      for (int row = 1; row <= n; row++) {
4          int distance = Math.min(row - 1, n - row);
5          int count = n - 2 * distance;
6          int value = (n + 1) / 2 + distance;
7          for (int sp = 1; sp <= distance * 2; sp++) {
8              System.out.print(s: " ");
9          }
10         for (int col = 1; col <= count; col++) {
11             System.out.print(value + " ");
12         }
13         System.out.println();
14     }
15 }
16 }
17 }
18

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

+ ▼ ... | [] X

● PS D:\oops with java> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Bhavya aggarwal\AppData\Roaming\Code\User\workspaceStorage\cac716e0002c014cf17c54c283732460\redhat.java\jdt_ws\oops with java_5dfb3bf8\bin' 'SandwichPattern'

```

3 3 3 3 3
 4 4 4
   5
 4 4 4
3 3 3 3 3

```

○ PS D:\oops with java>

Run: Inverte...

Run: Sandwi...

IncrementingDiamond.java > Java > IncrementingDiamond

```
1 public class IncrementingDiamond {  
    Run | Debug | Run main | Debug main  
2     public static void main(String[] args) {  
3         int n = 5;  
4         for (int i = 1; i <= 2 * n - 1; i++) {  
5             int k = (i <= n) ? i : 2 * n - i;  
6             int value = k + 2;  
7             for (int sp = 1; sp <= (n - k) * 2; sp++) {  
8                 System.out.print(s: " ");  
9             }  
10            for (int col = 1; col <= k; col++) {  
11                System.out.print(value + " ");  
12            }  
13            System.out.println();  
14        }  
15    }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

+ ▾ ... □ X

```
PS D:\oops with java> & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Bhavya aggarwal\AppData\Roaming\Code\User\workspaceStorage\cac716e0002c014cf17c54c283732\460\redhat.java\jdt_ws\oops with java_5dfb3bf8\bin' 'IncrementingDiamond'
```

Run: Inverte...

Run: Sandwi...

Run: Increm...

```
3  
4 4  
5 5 5  
6 6 6 6  
7 7 7 7 7  
6 6 6 6  
5 5 5  
4 4  
3
```